

Multiprocessing vs Multithreading

Multiprocessing

Multiprocessing involves using multiple processes to execute tasks concurrently. Each process has its own memory space, resources, and system context.

Key Characteristics:

- Each process runs in its own memory space
- Processes are isolated from each other
- Creating processes is more resource-intensive
- Better for CPU-intensive tasks
- Can utilize multiple CPU cores effectively

Example:

python

```
from multiprocessing import Process
import os

def worker(name):
    print(f"Process {name} - PID: {os.getpid()}")

if __name__ == "__main__":
    p1 = Process(target=worker, args=("A",))
    p2 = Process(target=worker, args=("B",))
    p1.start()
    p2.start()
    p1.join()
    p2.join()
```

Multithreading

Multithreading involves multiple threads within a single process. Threads share the same memory space and resources.

Key Characteristics:

- Threads share the same memory space
- Communication between threads is easier but can lead to race conditions
- Creating threads is lightweight
- Better for I/O-bound tasks
- In Python, limited by GIL (Global Interpreter Lock)

Example:

python

```
import threading
import time

def worker(name):
    print(f"Thread {name} starting")
    time.sleep(1)
    print(f"Thread {name} finished")

t1 = threading.Thread(target=worker, args=("A",))
t2 = threading.Thread(target=worker, args=("B",))
t1.start()
t2.start()
t1.join()
t2.join()
```

Comparison Table:

Aspect	Multiprocessing	Multithreading
Memory	Separate memory space	Shared memory space
Overhead	High	Low
Communication	Complex (IPC needed)	Simple (shared variables)
Best for	CPU-bound tasks	I/O-bound tasks

Crash impact	One process crashes, others continue	One thread crashes, all threads affected
--------------	--------------------------------------	--

Two Ways of Creating Threads

Method 1: Extending Thread Class

Create a subclass of `Thread` and override the `run()` method.

python

```
import threading

class MyThread(threading.Thread):
    def __init__(self, name):
        super().__init__()
        self.name = name

    def run(self):
        print(f"Thread {self.name} is running")
        # Thread logic here

# Usage
t = MyThread("CustomThread")
t.start()
t.join()
```

Method 2: Using Thread Constructor with Target Function

Pass a target function to the Thread constructor.

python

```
import threading

def worker_function(name):
    print(f"Thread {name} is running")
```

```
# Usage
t = threading.Thread(target=worker_function, args=("WorkerThread",))
t.start()
t.join()
```

Difference Between the Two Approaches:

Aspect	Extending Thread Class	Using Target Function
Complexity	More complex - requires class definition	Simpler - just a function
Reusability	Can create specialized thread classes	Reusable functions can be passed to any thread
Access to Thread Methods	Direct access using <code>self</code>	Limited to function parameters
Encapsulation	Better for complex thread logic	Better for simple tasks
Recommended Use	When thread has specific state/behavior	For simple, one-off tasks
Flexibility	Less flexible - fixed behavior	More flexible - can pass different functions

When to use each:

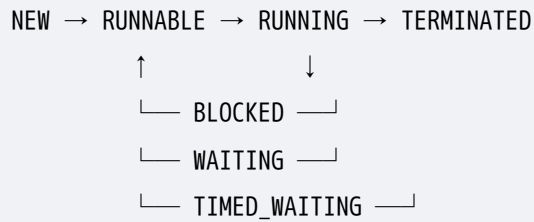
- Extending Thread: When you need specialized thread behavior with internal state
- Target Function: When you have a simple task or want to reuse existing functions

Thread Lifecycle

The thread lifecycle consists of several states a thread goes through:

State Diagram:

text



Detailed States:

1. NEW (Born State)

Thread is created but not yet started.

python

```
t = threading.Thread(target=some_function)
# Thread is in NEW state
```

2. RUNNABLE (Ready State)

Thread is ready to run and waiting for CPU time.

python

```
t.start()
# Thread moves to RUNNABLE state
```

3. RUNNING (Running State)

Thread is currently executing its code.

python

```
def my_function():
    # When this runs, thread is in RUNNING state
    pass
```

4. BLOCKED/WAITING/TIMED_WAITING

Thread is temporarily inactive:

- **BLOCKED:** Waiting for a lock (synchronization)
- **WAITING:** Waiting indefinitely for another thread
- **TIMED_WAITING:** Waiting for a specified time

python

```
def my_function():
    # BLOCKED - waiting for lock
    with some_lock:
        pass

    # WAITING - wait indefinitely
    event.wait()

    # TIMED_WAITING - wait for 5 seconds
    event.wait(timeout=5)
    time.sleep(2)
```

5. TERMINATED (Dead State)

Thread has finished execution or was interrupted.

python

```
# Thread automatically moves to TERMINATED after completion
```

Lifecycle Example:

python

```
import threading
import time

def worker():
    print("Thread started - RUNNING state")
    time.sleep(1) # TIMED_WAITING state
```

```
    print("Thread finishing")

# NEW state
t = threading.Thread(target=worker)

# RUNNABLE state
t.start()

# Thread runs through states
# Eventually TERMINATED state
t.join()
print("Thread terminated")

# Check if thread is alive
print(f"Thread alive: {t.is_alive()}") # False
```

Daemon Threads

A daemon thread is a background thread that runs in the background and does not prevent the program from exiting when the main program finishes.

Key Characteristics:

1. Background Execution: Runs in the background
2. Automatic Termination: Terminates when all non-daemon threads finish
3. No Cleanup: May be abruptly stopped without proper cleanup
4. Utility Tasks: Used for services, monitoring, garbage collection

Example - Daemon Thread:

python

```
import threading
import time

def background_task():
    count = 0
```

```

while True:
    print(f"Daemon thread running: {count}")
    count += 1
    time.sleep(1)

# Create daemon thread
daemon_thread = threading.Thread(target=background_task, daemon=True)
# OR using setDaemon() method
# daemon_thread.setDaemon(True)

daemon_thread.start()

# Main thread does some work
print("Main thread doing work...")
time.sleep(3)
print("Main thread finishing...")
# Program exits even though daemon thread is still running

```

Output:

text

```

Main thread doing work...
Daemon thread running: 0
Daemon thread running: 1
Daemon thread running: 2
Main thread finishing...
(Program exits, daemon thread is terminated)

```

Daemon vs Non-Daemon Threads:

python

```

import threading
import time

def regular_task():
    for i in range(5):
        print(f"Regular thread: {i}")
        time.sleep(1)

```



```
def daemon_task():
    count = 0
    while True:
        print(f"Daemon thread: {count}")
        count += 1
        time.sleep(0.5)

# Non-daemon thread (default)
regular = threading.Thread(target=regular_task)
# Daemon thread
daemon = threading.Thread(target=daemon_task, daemon=True)

regular.start()
daemon.start()

# Program will wait for regular thread to finish
# Daemon thread will be terminated when program exits
print("Main thread finishing...")
```

When to Use Daemon Threads:

Use Daemon Threads for:

- Background monitoring
- Logging
- Garbage collection
- Heartbeat services
- Cache refresh tasks

Avoid Daemon Threads for:

- Critical operations that need cleanup
- File writes that must complete
- Database operations
- Resource management

Best Practices:

```
import threading
import time

class GracefulDaemon:
    def __init__(self):
        self.running = True

    def run(self):
        while self.running:
            # Do background work
            time.sleep(1)

    def stop(self):
        self.running = False

def better_daemon():
    daemon = GracefulDaemon()
    t = threading.Thread(target=daemon.run, daemon=True)
    t.start()
    # Can implement proper shutdown if needed
    # daemon.stop()
```

Important Notes:

- Python programs exit when only daemon threads remain
- Daemon threads are abruptly terminated - no `finally` blocks execute
- Use `threading.current_thread().daemon` to check if thread is daemon
- Set daemon property before calling `start()`